

Entraîner un encodeur de recherche

Équipe Data Tigers • NLP 2 • M2 Data & IA • FGES

Fiche pédagogique : recherche dense, évaluation et apprentissage contrastif

Objet	Expliquer ce que fait l'encodeur pendant une recherche, ce qui distingue l'évaluation de l'entraînement, et comment l'apprentissage contrastif ajuste ses paramètres
Public	Toute l'équipe ; prolonge le support <i>Comprendre l'encodeur multilingual-e5-small</i> (sections 3 à 5), à lire d'abord
Usage	Préparer le notebook 03 (#46), qui évalue, et l'ajustement fin (#49), qui entraînerait
Version	6 octobre 2026
Vérifications	Chaque chiffre est soit cité d'une source primaire () , soit mesuré sur notre corpus ()

Les exemples numériques ont été calculés le 6 octobre 2026 sur les 4 240 passages du découpage M2, avec le modèle `intfloat/multilingual-e5-small` (révision 614241f6) en précision `fp32`. Les trois questions employées sont celles du notebook 02 : aucune ne vient du jeu de test. Elles illustrent un mécanisme ; elles ne mesurent pas la qualité du système.

Sommaire

1	L'essentiel en cinq points	3
2	Le modèle et ses paramètres	3
2.1	Une fonction des textes vers la sphère	3
2.2	Ce que contient θ	3
3	Chercher : une ligne de matrice, puis un tri	4
3.1	Le calcul fait à chaque question	4
3.2	La matrice de similarités	4
3.3	Pourquoi ne pas tenir une table de scores	4
3.4	Bi-encodeur et modèle croisé	5
4	Évaluer n'est pas entraîner	5
5	Entraîner : la matrice d'un lot	5
5.1	Les données et la construction du lot	5
5.2	Un lot réel sur notre corpus	6

5.3 Pourquoi les négatifs sont indispensables	6
6 La perte InfoNCE	6
6.1 Définition	6
6.2 D'où vient le nom	6
6.3 Bornes	7
6.4 Le rôle de la température	7
7 Le gradient : qui pousse qui	8
7.1 Par rapport aux scores	8
7.2 Jusqu'aux vecteurs	8
7.3 Jusqu'aux paramètres : la rétropropagation	9
8 Optimiser : la descente de gradient	9
8.1 Le pas	9
8.2 Les ordres de grandeur pour nous	9
8.3 Deux risques	9
9 Ce que cela implique pour notre projet	10
9.1 Où nous en sommes	10
9.2 Une recette pour l'ajustement fin (#49)	10
10 Glossaire	10
11 Sources	11

1 L'essentiel en cinq points

1. **Le modèle est une fonction** qui transforme *un* texte en *un* vecteur de 384 nombres. Il encode chaque texte séparément, question ou passage ; il ne voit jamais un couple d'un seul bloc.
2. **La matrice de similarités n'est pas le modèle** : c'est un calcul fait à partir de ses vecteurs. Chercher, c'est calculer une ligne de cette matrice pour la question posée, puis trier.
3. **Évaluer n'est pas entraîner**. Les notebooks 01 à 03 et le site utilisent le modèle sans le modifier ; le notebook 03 mesure son classement sur nos questions.
4. **Entraîner, c'est modifier ses 117,7 millions de paramètres** pour que, dans tout lot d'exemples, chaque question soit plus proche de son passage que des autres passages du lot. La perte InfoNCE mesure l'écart à cet objectif ; la rétropropagation dit comment chaque paramètre doit bouger.
5. **Le modèle a déjà été entraîné** par ses auteurs. L'ajustement fin (#49) reprendrait le même mécanisme sur des couples de notre domaine, distincts du jeu de test, puis exigerait de recalculer l'index et de remesurer.

2 Le modèle et ses paramètres

2.1 Une fonction des textes vers la sphère

Notons $\mathcal{T}$ l'ensemble des textes et $d = 384$. L'encodeur est une fonction paramétrée

$$f_{\theta} : \mathcal{T} \longrightarrow \mathbb{S}^{d-1} = \{\mathbf{x} \in \mathbb{R}^d : \|\mathbf{x}\|_2 = 1\},$$

où θ désigne l'ensemble de ses paramètres. Pour un texte x , le réseau produit un vecteur $\mathbf{h}_t \in \mathbb{R}^d$ par token, puis

$$\mathbf{h}(x) = \frac{\sum_t m_t \mathbf{h}_t}{\sum_t m_t}, \quad f_{\theta}(x) = \frac{\mathbf{h}(x)}{\|\mathbf{h}(x)\|_2},$$

avec $m_t = 1$ pour un token réel et 0 pour un remplissage (support sur l'encodeur, section 3.4). Une question q et un passage p passent dans le **même** réseau, distingués par un préfixe  :

$$\mathbf{u} = f_{\theta}(\text{query: } q), \quad \mathbf{e} = f_{\theta}(\text{passage: } p).$$

2.2 Ce que contient θ

Composant	Paramètres	Part
Table des tokens : 250 037 lignes de 384 nombres (le tokeniseur en emploie 250 002)	96 014 208	81,6 %
12 couches Transformer : 12 têtes d'attention, dimension 384, couche intermédiaire de 1536	21 293 568	18,1 %
Positions (512×384), types de segments et normalisation de la table	198 144	0,2 %
Couche <i>pooler</i> , que la moyenne des tokens n'utilise pas	147 840	0,1 %
Total	117 653 760	100 %

🔍 Décompte fait sur le modèle chargé. Une couche compte 1 774 464 paramètres : $4 \times (384^2 + 384)$ pour l'attention, $2(384 \times 1\,536) + 1\,536 + 384$ pour la couche intermédiaire, et 2×768 pour ses deux normalisations. Les quatre cinquièmes du modèle servent donc à donner un vecteur de départ à chaque token du vocabulaire multilingue : c'est ce que vise la réduction du vocabulaire (#50).

3 Chercher : une ligne de matrice, puis un tri

3.1 Le calcul fait à chaque question

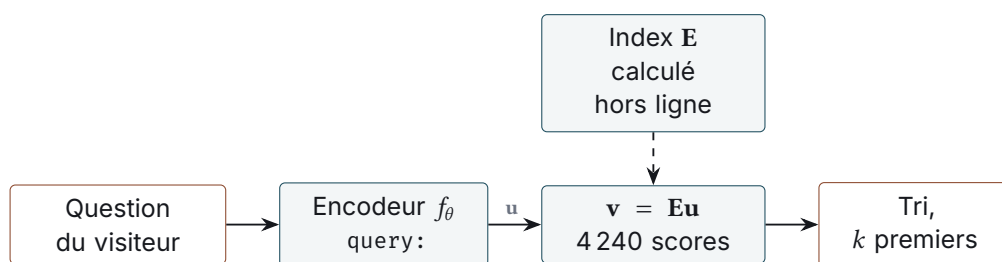
Le notebook 02 a calculé une fois pour toutes la matrice des passages, ou **index**,

$$\mathbf{E} = \begin{pmatrix} \mathbf{e}_1^\top \\ \vdots \\ \mathbf{e}_N^\top \end{pmatrix} \in \mathbb{R}^{N \times d}, \quad N = 4\,240,$$

soit $4\,240 \times 384 \times 4$ octets = 6,21 Mio en fp32 📊. Pour une question q de vecteur $\mathbf{u}$, le site calcule

$$\mathbf{v} = \mathbf{E} \mathbf{u} \in \mathbb{R}^N, \quad v_j = \langle \mathbf{e}_j, \mathbf{u} \rangle = \cos(\mathbf{e}_j, \mathbf{u}),$$

puis classe les passages par score décroissant, les ex aequo dans l'ordre de l'index, et affiche les k premiers. Le calcul de $\mathbf{v}$ coûte $Nd = 1\,628\,160$ multiplications et autant d'additions : 1,7 ms en médiane dans le navigateur, pour environ 20 ms d'encodage de la question 📊.



3.2 La matrice de similarités

Pour évaluer, on pose d'un coup n questions de vecteurs $\mathbf{u}_1, \dots, \mathbf{u}_n$, réunis dans $\mathbf{U} \in \mathbb{R}^{n \times d}$, et l'on forme

$$\mathbf{S} = \mathbf{U} \mathbf{E}^\top \in \mathbb{R}^{n \times N}, \quad S_{ij} = \langle \mathbf{u}_i, \mathbf{e}_j \rangle.$$

Chaque ligne est la recherche d'une question. Le notebook 03 construit cette matrice pour les questions du jeu de test et y lit le rang du premier passage pertinent. Le site, lui, ne la stocke jamais : il calcule à la volée la seule ligne de la question posée.

3.3 Pourquoi ne pas tenir une table de scores

On pourrait imaginer d'apprendre directement les scores, une case par couple (question, passage). Une telle table n'est définie que pour les questions qu'elle contient : pour une question nouvelle, il n'existe aucune ligne. Or l'ensemble des questions possibles est illimité, et presque toutes celles du site seront nouvelles. L'encodeur, au contraire, est défini sur *tout* texte, avec un nombre de paramètres fixé qui ne dépend pas du nombre de questions : deux formulations voisines reçoivent des vecteurs voisins, ce qui lui permet de **généraliser**. Une table apprise serait la forme extrême du surapprentissage : une mémoire, sans généralisation.

3.4 Bi-encodeur et modèle croisé

Un modèle **croisé** lit la question et le passage ensemble, $g_\theta(q, p) \in \mathbb{R}$, et classe souvent mieux. Mais il ne permet aucun calcul préalable : chaque question exigerait $N = 4\,240$ passages complets dans le réseau. Au coût d'encodage d'une seule question, environ 20 ms, cela fait déjà plus de 80 secondes par question, et davantage en réalité, car un couple est bien plus long qu'une question. Le **bi-encodeur** déplace tout le travail lourd hors ligne : c'est ce qui rend le volet A possible dans un navigateur (support sur l'encodeur, section 5.1).

4 Évaluer n'est pas entraîner

	Inférence et évaluation	Entraînement
Paramètres θ	fixes	modifiés à chaque pas
Données	n'importe quel texte ; pour évaluer, des questions annotées	des couples (question, passage qui répond)
Résultat	des vecteurs, des classements, des métriques	de nouveaux paramètres θ'
Dans notre projet	notebooks 01, 02 et 03, site du volet A	ajustement fin (#49), non commencé

Évaluer consiste à lire la matrice $\mathbf{S}$. Pour la question i , soit r_i le rang de son premier passage pertinent ; avec n questions,

$$\text{Recall}@k = \frac{1}{n} \sum_{i=1}^n \mathbf{1}[r_i \leq k], \quad \text{MRR}@10 = \frac{1}{n} \sum_{i=1}^n \frac{\mathbf{1}[r_i \leq 10]}{r_i}.$$

Ces définitions sont celles de `scripts/evaluation/metrics.py`, validées sur PIAF (#45). Rien n'y est appris.

BM25 n'apprend rien non plus. Sa méthode `fit()` compte, pour chaque mot, le nombre de passages qui contiennent, et mesure la longueur de chaque passage : ce sont des statistiques, sans gradient. Ses deux réglages, k_1 et b , sont fixés à la main. Le nom `fit` vient seulement de la convention de scikit-learn.

5 Entraîner : la matrice d'un lot

5.1 Les données et la construction du lot

On dispose d'un ensemble de couples $\mathcal{D} = \{(q_i, p_i^+)\}$, où p_i^+ répond à q_i . Un pas d'entraînement tire un **lot** de n couples, les encode, et forme la matrice carrée

$$\mathbf{S}^B \in \mathbb{R}^{n \times n}, \quad S_{ij} = \langle \mathbf{u}_i, \mathbf{e}_j \rangle, \quad \mathbf{u}_i = f_\theta(\text{query: } q_i), \quad \mathbf{e}_j = f_\theta(\text{passage: } p_j^+).$$

Les couples étant rangés dans le même ordre en lignes et en colonnes, **les bons couples sont sur la diagonale par construction**. Pour la question i , les autres colonnes contiennent les passages des autres couples : ce sont les **négatifs du lot**, obtenus sans aucune annotation supplémentaire. C'est la stratégie de DPR et d'E5 .

5.2 Un lot réel sur notre corpus

	p_1 mentions interdites	p_2 arrêt maladie	p_3 essai, CDI	p_4 essai, CDD
q_1 : Quelles mentions sont interdites sur le bulletin de paie ?	0,902	0,800	0,797	0,802
q_2 : Mon employeur peut-il me licencier pendant un arrêt maladie ?	0,811	0,885	0,805	0,808
q_3 : Combien de temps peut durer la période d'essai d'un CDI ?	0,787	0,828	0,884	0,893

 Cosinus du modèle actuel. Les colonnes p_1 à p_3 forment le lot 3×3 ; p_4 , la section de la fiche CDD sur la durée de la période d'essai, est ajouté comme **négatif difficile** pour q_3 . C'est l'échec observé au notebook 02 : p_4 (0,893) devance p_3 (0,884).

5.3 Pourquoi les négatifs sont indispensables

Si l'on cherchait seulement à rapprocher chaque question de son passage, la fonction constante serait une solution optimale : tous les vecteurs égaux donnent $S_{ii} = 1$, le cosinus maximal, et le modèle ne distingue plus rien. C'est l'**effondrement**. Les négatifs rendent l'objectif *relatif* : il ne suffit plus d'être proche de son passage, il faut l'être *plus* que des autres. La proposition 2 montre que la perte InfoNCE pénalise l'effondrement.

6 La perte InfoNCE

6.1 Définition

Avec une température $\tau > 0$, on pose pour chaque ligne i du lot

$$P_{ij} = \frac{\exp(S_{ij}/\tau)}{\sum_{k=1}^n \exp(S_{ik}/\tau)}, \quad \ell_i = -\log P_{ii}, \quad \mathcal{L} = \frac{1}{n} \sum_{i=1}^n \ell_i.$$

$P_{i\cdot}$ est une loi de probabilité sur les n passages du lot (fonction *softmax*) ; ℓ_i est l'entropie croisée de la classification « parmi ces n passages, lequel est le bon ? ». Des négatifs difficiles s'ajoutent simplement au dénominateur ; E5 en emploie 7 par exemple lors de son ajustement fin . En réécrivant ℓ_i ,

$$\ell_i = \log \sum_{k=1}^n \exp\left(\frac{S_{ik} - S_{ii}}{\tau}\right),$$

on voit que la perte ne dépend que des **écarts** entre le score du bon passage et ceux des autres.

6.2 D'où vient le nom

L'estimation contrastive par bruit (*Noise-Contrastive Estimation*) apprend un modèle en le faisant distinguer de vraies données d'échantillons de bruit . InfoNCE en est la variante de van den Oord et al. : sous leurs

hypothèses, des négatifs tirés indépendamment, minimiser $\mathcal{L}$ maximise une borne inférieure de l'information mutuelle entre questions et passages, $I(q; p) \geq \log n - \mathcal{L}$ . D'où l'intérêt de lots nombreux : E5 a été pré-entraîné avec des lots de 32 768 couples .

6.3 Bornes

Proposition 1. Pour des vecteurs normalisés, donc des scores dans $[-1, 1]$, chaque terme vérifie $\ell_i \geq \log(1 + (n-1)e^{-2/\tau}) > 0$.

Démonstration. P_{ii} est maximal quand $S_{ii} = 1$ et $S_{ik} = -1$ pour $k \neq i$; alors $P_{ii} = e^{1/\tau} / (e^{1/\tau} + (n-1)e^{-1/\tau}) = 1 / (1 + (n-1)e^{-2/\tau})$, et $\ell_i = -\log P_{ii}$. 

Avec $\tau = 0,01$ et $n = 32$, cette borne vaut $\log(1 + 31e^{-200})$, soit pratiquement zéro : la perte peut devenir aussi petite que l'on veut, sans jamais s'annuler.

Proposition 2. Si f_θ est constante, alors $\mathcal{L} = \log n$, quelle que soit τ .

Démonstration. Tous les vecteurs sont égaux, donc $S_{ik} = 1$ pour tous i, k , puis $P_{ii} = 1/n$ et $\ell_i = \log n$. 

Pour $n = 32$, l'effondrement coûte $\log 32 \approx 3,466$, très loin du minimum : il n'est donc pas une solution de la minimisation.

6.4 Le rôle de la température

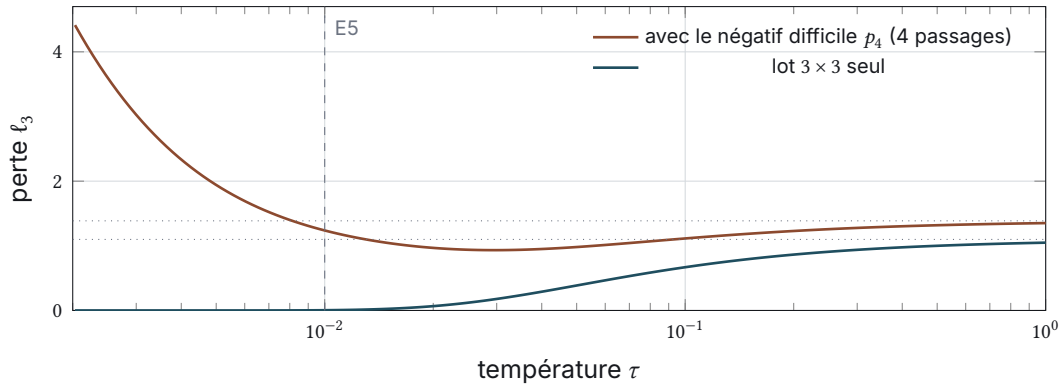
Comme $\ell_i = \log \sum_k \exp((S_{ik} - S_{ii})/\tau)$, deux limites éclairent son rôle :

- quand $\tau \rightarrow 0$, la somme est dominée par son plus grand terme, et $\ell_i \approx \max(0, \max_{k \neq i} S_{ik} - S_{ii})/\tau$: seule compte la question de savoir si le plus dangereux des négatifs passe devant le bon passage, et de combien ;
- quand $\tau \rightarrow \infty$, tous les termes tendent vers 1, P_i devient uniforme et $\ell_i \rightarrow \log n$: le signal d'apprentissage disparaît.

E5 emploie $\tau = 0,01$ . C'est cohérent avec ses scores resserrés : deux passages pris au hasard sur notre corpus ont un cosinus médian de 0,834  (support sur l'encodeur, section 4.4). Sans une petite température, des écarts de quelques centièmes resteraient sans effet.

Lot réel de la section 5.2	$\tau = 0,01$ (E5)	$\tau = 0,05$
Lot 3×3 : probabilité du bon passage pour q_1, q_2, q_3	$> 0,999$; 0,999 ; 0,996	0,798 ; 0,698 ; 0,680
Lot 3×3 : perte $\mathcal{L}$	0,002	0,324
Avec le négatif difficile p_4 : probabilité du bon passage pour q_3	0,290	0,375
Avec le négatif difficile p_4 : perte ℓ_3	1,237	0,981

 Le lot 3×3 est déjà résolu par le modèle actuel : avec $\tau = 0,01$, sa perte est presque nulle, et un tel lot n'apprendrait presque rien. Le négatif difficile change tout : q_3 ne place plus que 29 % de sa probabilité sur le bon passage, et sa perte passe de 0,004 à 1,237.



 Perte de q_3 en fonction de τ , calculée à partir des scores exacts du lot de la section 5.2. Les pointillés marquent $\log 4$ et $\log 3$, vers lesquelles chaque courbe tend quand τ grandit. Quand τ diminue, la perte s'annule si le bon passage est en tête (courbe bleue), et croît comme $0,009/\tau$ si un négatif le devance de 0,009 (courbe brune).

7 Le gradient : qui pousse qui

7.1 Par rapport aux scores

Proposition 3. $\frac{\partial \ell_i}{\partial S_{ij}} = \frac{1}{\tau} (P_{ij} - \mathbf{1}[i = j])$.

Démonstration. $\ell_i = -S_{ii}/\tau + \log \sum_k \exp(S_{ik}/\tau)$. La dérivée du premier terme vaut $-\mathbf{1}[i = j]/\tau$; celle du second vaut $\exp(S_{ij}/\tau) / (\tau \sum_k \exp(S_{ik}/\tau)) = P_{ij}/\tau$. $\square$

La descente de gradient déplace chaque score dans le sens opposé à sa dérivée. Le bon score S_{ii} est donc **tiré vers le haut** avec un poids $(1 - P_{ii})/\tau$, et chaque négatif **repoussé** avec un poids P_{ij}/τ . Comme $\sum_j P_{ij} = 1$, la traction exercée sur le bon passage est exactement répartie sur les négatifs, **au prorata de leur probabilité**. Un négatif facile ($P_{ij} \approx 0$) ne reçoit presque rien; un négatif difficile reçoit presque tout.

 Sur le lot réel avec $\tau = 0,01$, la ligne de q_3 vaut $P_3 = (0,000; 0,001; 0,290; 0,709)$. Le bon passage p_3 est tiré avec le poids $1 - 0,290 = 0,710$; le passage CDD en reçoit 0,709, soit la quasi-totalité. Les passages sur le bulletin de paie et la maladie, faciles, ne reçoivent pratiquement rien : un lot de négatifs aléatoires apprend peu.

7.2 Jusqu'aux vecteurs

Comme $S_{ij} = \langle \mathbf{u}_i, \mathbf{e}_j \rangle$, on a $\partial S_{ij} / \partial \mathbf{u}_i = \mathbf{e}_j$ et $\partial S_{ij} / \partial \mathbf{e}_j = \mathbf{u}_i$, donc

$$\frac{\partial \ell_i}{\partial \mathbf{u}_i} = \frac{1}{\tau} \left(\sum_j P_{ij} \mathbf{e}_j - \mathbf{e}_i \right).$$

Un pas de gradient déplace $\mathbf{u}_i$ vers $\mathbf{e}_i$ et l'éloigne de la moyenne des passages pondérée par P_i , c'est-à-dire surtout des négatifs dangereux. Symétriquement, chaque $\mathbf{e}_j$ bouge.

Ces vecteurs restent sur la sphère, car ils sont normalisés. Avec $f = \mathbf{h} / \|\mathbf{h}\|$, la dérivée de la normalisation vaut

$$\frac{\partial f}{\partial \mathbf{h}} = \frac{1}{\|\mathbf{h}\|} (\mathbf{I} - f f^\top),$$

qui projette le gradient sur le plan tangent à la sphère : l'apprentissage fait *tourner* les vecteurs, il ne les allonge pas.

7.3 Jusqu'aux paramètres : la rétropropagation

Les vecteurs dépendent de θ à travers la normalisation, la moyenne des tokens ($\partial \mathbf{h} / \partial \mathbf{h}_t = m_t / \sum_s m_s$), les 12 couches et la table des tokens. La **rétropropagation** applique la règle de dérivation en chaîne de la perte vers l'entrée, couche par couche, et fournit en une seule passe arrière la dérivée de $\mathcal{L}$ par rapport à chacun des 117 653 760 paramètres. Comme questions et passages passent dans le même réseau, les deux côtés contribuent au gradient des mêmes paramètres : un pas déplace à la fois les vecteurs des questions et ceux des passages.

8 Optimiser : la descente de gradient

8.1 Le pas

La descente de gradient stochastique remplace θ par

$$\theta_{t+1} = \theta_t - \eta \mathbf{g}_t, \quad \mathbf{g}_t = \nabla_{\theta} \mathcal{L}(\theta_t; B_t),$$

où B_t est le lot du pas t et η le pas d'apprentissage. Le gradient d'un lot est une estimation bruitée du gradient sur toutes les données ; c'est le sens de « stochastique ». En pratique, on emploie AdamW, qui adapte le pas de chaque paramètre à l'aide de moyennes mobiles des gradients et de leurs carrés, et applique une décroissance des poids découplée .

8.2 Les ordres de grandeur pour nous

 Sur la machine de référence, un pas sur un lot de 32 couples prend 89 ms (journal d'usage de l'IA, session 8). Avec 17 106 couples, une époque compte $\lceil 17\,106/32 \rceil = 535$ pas, et trois époques 1 605 pas, soit environ 143 secondes : 2,4 minutes. Le coût de calcul n'est pas un obstacle. Avec des lots de 32, chaque question n'a que 31 négatifs, contre 32 767 pour E5 ; les auteurs notent que des négatifs difficiles permettent de compenser des lots plus petits .

8.3 Deux risques

- Le **surapprentissage** : à force de pas, le modèle apprend par cœur les couples d'entraînement et généralise moins bien. Parades : un petit pas η , peu d'époques, et l'arrêt quand la perte d'un ensemble de validation, distinct de l'entraînement et du test, cesse de baisser.
- L'**oubli catastrophique** : en s'adaptant à notre domaine, le modèle peut perdre une partie de ce qu'il savait du français général. Seule une mesure avant et après, sur le même jeu figé, permet de le voir.

9 Ce que cela implique pour notre projet

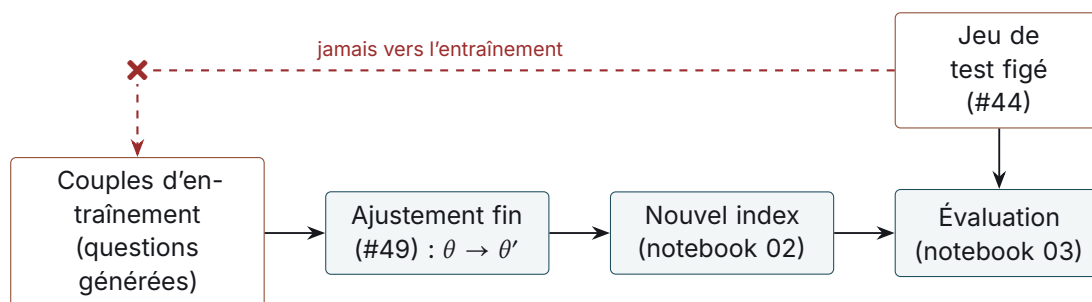
9.1 Où nous en sommes

Le modèle a été entraîné par ses auteurs : pré-entraînement contrastif sur environ un milliard de couples de textes, puis ajustement fin sur environ 1,6 million de couples annotés, avec négatifs difficiles et distillation depuis un modèle croisé 📖. Nous l'utilisons tel quel. Le notebook 03 fournira la mesure **avant** tout ajustement.

9.2 Une recette pour l'ajustement fin (#49)

Proposition, à décider dans le ticket :

- 1. Données.** Des couples (question, passage qui répond) construits sur les passages M2, par exemple avec des questions générées par IA, que le README du jeu de questions réserve à l'entraînement. **Jamais une question du jeu de test**, sans quoi la mesure après ajustement évaluerait de la mémoire.
- 2. Validation.** Un ensemble de validation tiré des couples d'entraînement, séparé par fiche, pour arrêter l'entraînement à temps.
- 3. Lots sans faux négatifs.** Des lots composés de fiches différentes : deux couples de la même fiche feraient d'un passage pertinent un négatif. Le découpage M2 compte 57 passages à texte identique 📖 (support sur l'encodeur, section 8.4).
- 4. Négatifs difficiles.** Pour chaque question, des passages que le modèle actuel classe haut sans qu'ils répondent, vérifiés : le passage CDD pour une question sur le CDI en est l'exemple type.
- 5. Après l'entraînement.** f_θ a changé : recalculer l'index des 4 240 passages (notebook 02), réexporter le modèle en ONNX q8 et contrôler sa fidélité dans le navigateur, puis remesurer avec le notebook 03, selon le même protocole et sur le même jeu figé.



10 Glossaire

Encodeur fonction f_θ qui transforme un texte en vecteur normalisé.

Paramètres les nombres θ du réseau, 117 653 760 ici, que l'entraînement modifie.

Inférence utilisation du modèle à paramètres fixes : encoder, chercher, évaluer.

Lot ensemble de n couples traités ensemble à un pas d'entraînement.

Négatif du lot passage d'un autre couple du lot, employé comme mauvaise réponse.

Négatif difficile passage proche de la question sans y répondre, choisi exprès.

Faux négatif passage traité comme négatif alors qu'il répond à la question.

Softmax transformation de scores en probabilités : $P_{ij} \propto \exp(S_{ij}/\tau)$.

Température constante τ qui règle la sévérité de la comparaison des scores.

InfoNCE perte $-\log P_{ii}$ moyennée sur le lot : entropie croisée de la classification du bon passage.

Effondrement solution triviale où tous les textes reçoivent le même vecteur.

Gradient vecteur des dérivées de la perte par rapport aux paramètres.

Rétropropagation calcul de ce gradient par dérivation en chaîne, de la perte vers l'entrée.

Époque passage complet sur les données d'entraînement.

Surapprentissage apprentissage par cœur des exemples, au détriment de la généralisation.

Oubli catastrophique perte de compétences générales lors d'une adaptation à un domaine.

Bi-encodeur question et passage encodés séparément puis comparés ; passages encodés à l'avance.

Modèle croisé question et passage lus ensemble ; plus précis, sans calcul préalable possible.

11 Sources

- Gutmann M. et Hyvärinen A. (2010), *Noise-contrastive estimation: A new estimation principle for unnormalized statistical models*, AISTATS.
- van den Oord A., Li Y. et Vinyals O. (2018), *Representation Learning with Contrastive Predictive Coding*, arXiv:1807.03748.
- Karpukhin V. et al. (2020), *Dense Passage Retrieval for Open-Domain Question Answering*, EMNLP : négatifs du lot.
- Loshchilov I. et Hutter F. (2019), *Decoupled Weight Decay Regularization*, ICLR : AdamW.
- Wang L. et al. (2022), *Text Embeddings by Weakly-Supervised Contrastive Pre-training*, arXiv:2212.03533, sections 4.1 et 4.2 et section 5 : perte, $\tau = 0,01$, préfixes, négatifs du lot, lots de 32 768, 7 négatifs difficiles.
- Wang L. et al. (2024), *Multilingual E5 Text Embeddings: A Technical Report*, arXiv:2402.05672, tableaux 1 et 2 : données de pré-entraînement et d'ajustement fin.
- Équipe Data Tigers : support *Comprendre l'encodeur multilingual-e5-small* ; rapport de validation des métriques sur PIAF ; journal d'usage de l'IA, session 8.